

N4_Neteja_i_pre_processat_de_dades

November 26, 2025

1 Neteja i pre-processat de dades

```
[ ]: !wget https://raw.githubusercontent.com/I-Onna/dades/refs/heads/main/
      ↪alumnes_tractament_dades_v2.csv
```

```
--2025-11-21 16:12:41-- https://raw.githubusercontent.com/I-
Onna/dades/refs/heads/main/alumnes_tractament_dades_v2.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
185.199.110.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 15570 (15K) [text/plain]
Saving to: 'alumnes_tractament_dades_v2.csv'

      alumnes_t    0%[                               ]      0  --.-KB/s
alumnes_tractament_ 100%[=====>]  15.21K  --.-KB/s    in 0s

2025-11-21 16:12:41 (55.3 MB/s) - 'alumnes_tractament_dades_v2.csv' saved
[15570/15570]
```

```
[ ]: import pandas as pd

df = pd.read_csv("alumnes_tractament_dades_v2.csv")
df.head()
```

```
[ ]:  ID Alumne      Nom  Edat  Alçada (m)  Programació  Bases de Dades  \
0  ALU-1115  Cirino Acevedo   25      1.74          7.3          3.7
1  ALU-1032  Amada Bermudez   18      1.85          6.7          5.6
2  ALU-1214   Pánfilo Diez   23      1.72          6.7          4.8
3  ALU-1003   Pastor Reyes   22      1.76          7.0          8.4
4  ALU-1068  Estela Escobar   18      1.80          6.4          6.0

      Sistemes Operatius  Desenvolupament Web  Hores d'Estudi  \
0                5.0                4.8        15.0
1                3.0                9.1        14.0
2               10.0                6.9         9.0
```

3	3.7	10.0	20.0
4	10.0	8.0	19.0

	Faltes d'Assistència	Grup
0	0	DAWA
1	7	DAWB
2	19	DAWB
3	0	DAMA
4	0	DAMB

1.0.1 Modificar el nom de les columnes

Modificar el nom de les columnes en un DataFrame és una pràctica molt comuna per a un maneig més clar, eficaç i intuïtiu de les dades, especialment quan s'elaboren processos d'anàlisi o transformació.

Canviar els noms de les columnes d'un DataFrame ens permet tenir noms més significatius que expressin clarament la informació que contenen, per simplificar noms massa llargs, per adaptar-los a estàndards o convencions del projecte o per evitar conflictes o duplicats que puguin afectar la manipulació de dades.

```
df.columns = ['Nom1', 'Nom2', 'Nom3', ...]
```

Podem assignar una nova llista completa de noms però perquè Perquè això funcioni correctament, la llista amb els nous noms ha de tenir exactament la mateixa longitud que el nombre de columnes existents al DataFrame. Aquest mètode és especialment recomanat quan es vol canviar tots els noms d'una manera consistent i ordenada.

```
[ ]: df.head()
```

```
[ ]: ID Alumne      Nom      Edat      Alçada (m)      Programació      Bases de Dades \
0  ALU-1115  Cirino Acevedo      25          1.74          7.3          3.7
1  ALU-1032  Amada Bermudez      18          1.85          6.7          5.6
2  ALU-1214   Pánfilo Diez      23          1.72          6.7          4.8
3  ALU-1003   Pastor Reyes      22          1.76          7.0          8.4
4  ALU-1068   Estela Escobar      18          1.80          6.4          6.0
```

	Sistemes Operatius	Desenvolupament Web	Hores d'Estudi \
0	5.0	4.8	15.0
1	3.0	9.1	14.0
2	10.0	6.9	9.0
3	3.7	10.0	20.0
4	10.0	8.0	19.0

	Faltes d'Assistència	Grup
0	0	DAWA
1	7	DAWB
2	19	DAWB
3	0	DAMA

```
[ ]: df.columns = ['Id_alumne', 'Nom_complert', 'Edat', 'Alçada', 'PED', 'BBDD', 'SO', 'DIW', 'Hores_estudi', 'Faltes_assistència', 'Grup_classe']
df.head()
```

```
[ ]:   Id_alumne  Nom_complert  Edat  Alçada  PED  BBDD   SO  DIW  \
0  ALU-1115  Cirino Acevedo   25   1.74  7.3   3.7   5.0  4.8
1  ALU-1032  Amada Bermudez   18   1.85  6.7   5.6   3.0  9.1
2  ALU-1214  Pánfilo Diez    23   1.72  6.7   4.8  10.0  6.9
3  ALU-1003  Pastor Reyes    22   1.76  7.0   8.4   3.7  10.0
4  ALU-1068  Estela Escobar   18   1.80  6.4   6.0  10.0  8.0

      Hores_estudi  Faltes_assistència  Grup_classe
0              15.0                  0          DAWA
1              14.0                  7          DAWB
2               9.0                 19          DAWB
3              20.0                  0          DAMA
4              19.0                  0          DAMB
```

La funció `rename()` de pandas permet canviar noms de columnes selectivament passant un diccionari on les claus són els noms antics i els valors els noms nous.

```
df.rename(columns={'antic_nom1':'nou_nom1', 'antic_nom2':'nou_nom2'}, inplace=True)
```

Amb l'argument `inplace=True` podem aplicar el canvi directament al DataFrame original sense haver de crear una còpia.

```
[ ]: df.rename(columns={'Id_alumne':'ALU-ID'}, inplace=True)
df.head()
```

```
[ ]:   ALU-ID  Nom_complert  Edat  Alçada  PED  BBDD   SO  DIW  \
0  ALU-1115  Cirino Acevedo   25   1.74  7.3   3.7   5.0  4.8
1  ALU-1032  Amada Bermudez   18   1.85  6.7   5.6   3.0  9.1
2  ALU-1214  Pánfilo Diez    23   1.72  6.7   4.8  10.0  6.9
3  ALU-1003  Pastor Reyes    22   1.76  7.0   8.4   3.7  10.0
4  ALU-1068  Estela Escobar   18   1.80  6.4   6.0  10.0  8.0

      Hores_estudi  Faltes_assistència  Grup_classe
0              15.0                  0          DAWA
1              14.0                  7          DAWB
2               9.0                 19          DAWB
3              20.0                  0          DAMA
4              19.0                  0          DAMB
```

1.0.2 Eliminar valors duplicats

Detectar i tractar files duplicades és un pas molt important en la neteja i preparació de dades.

- **df.duplicated():** Retorna una sèrie booleana que indica quines files estan duplicades respecte a files anteriors (per defecte manté la primera aparició).

```
[ ]: df[df.duplicated()]
```

```
[ ]: Empty DataFrame
Columns: [ALU-ID, Nom_complert, Edat, Alçada, PED, BBDD, SO, DIW, Hores_estudi,
Faltes_assistència, Grup_classe]
Index: []
```

També podem utilitzar el paràmetre subset per restringir la detecció a unes columnes concretes que defineixen la duplicació (per exemple, clau primària o identificador):

```
[ ]: df[df.duplicated(subset=['ALU-ID'])]
```

```
[ ]:
      ALU-ID  Nom_complert  Edat  Alçada  PED  BBDD   SO  DIW  \
35  ALU-1142  Macario Galiano   20    1.73  7.3   1.1  10.0  6.1
123 ALU-1097   Eusebia Boix   20    1.63  7.0   8.2   4.7  4.5
200 ALU-1006  Loreto Quesada   18    1.74  5.7   5.8   3.8  6.9
212 ALU-1112   Felix Bayona   32    1.65  7.1   7.8   4.1  7.4
233 ALU-1060  Adán Caparrós   19    1.82  6.1   2.5   7.7  7.5

      Hores_estudi  Faltes_assistència  Grup_classe
35              11.0                11         NaN
123             25.0                 0         DAMB
200             26.0                 1         NaN
212              9.0                25         DAWB
233              9.0                 2         NaN
```

```
[ ]: df[df.duplicated(subset=['ALU-ID', 'Nom_complert'])]
```

```
[ ]:
      ALU-ID  Nom_complert  Edat  Alçada  PED  BBDD   SO  DIW  \
35  ALU-1142  Macario Galiano   20    1.73  7.3   1.1  10.0  6.1
123 ALU-1097   Eusebia Boix   20    1.63  7.0   8.2   4.7  4.5
200 ALU-1006  Loreto Quesada   18    1.74  5.7   5.8   3.8  6.9
212 ALU-1112   Felix Bayona   32    1.65  7.1   7.8   4.1  7.4
233 ALU-1060  Adán Caparrós   19    1.82  6.1   2.5   7.7  7.5

      Hores_estudi  Faltes_assistència  Grup_classe
35              11.0                11         NaN
123             25.0                 0         DAMB
200             26.0                 1         NaN
212              9.0                25         DAWB
233              9.0                 2         NaN
```

Amb el paràmetre keep=False veurem totes les aparicions dels registres duplicats.

```
[ ]: df[df.duplicated(subset=['ALU-ID'], keep=False)]
```

```
[ ]:      ALU-ID      Nom_complert  Edat  Alçada  PED  BBDD      SO  DIW  \
18  ALU-1006  Loreto Quesada    18    1.74  NaN   5.8    3.8  6.9
25  ALU-1142  Macario Galiano    20    1.73  7.3    1.1   10.0  6.1
26  ALU-1097   Eusebia Boix    20    1.63  7.0    8.2    4.7  4.5
35  ALU-1142  Macario Galiano    20    1.73  7.3    1.1   10.0  6.1
123 ALU-1097   Eusebia Boix    20    1.63  7.0    8.2    4.7  4.5
179 ALU-1112   Felix Bayona    32    1.65  7.1    7.8    4.1  7.4
200 ALU-1006  Loreto Quesada    18    1.74  5.7    5.8    3.8  6.9
212 ALU-1112   Felix Bayona    32    1.65  7.1    7.8    4.1  7.4
226 ALU-1060  Adán Caparrós    19    1.82  6.1    2.5    7.7  7.5
233 ALU-1060  Adán Caparrós    19    1.82  6.1    2.5    7.7  7.5
```

```
      Hores_estudi  Faltes_assistència  Grup_classe
18              26.0                1      DAWB
25              11.0               11      DAMA
26              25.0                0      NaN
35              11.0               11      NaN
123             25.0                0      DAMB
179              9.0               25      NaN
200             26.0                1      NaN
212              9.0               25      DAWB
226              9.0                2      DAWB
233              9.0                2      NaN
```

La funció `drop_duplicates()` de pandas és la manera principal i més senzilla d'eliminar files duplicades en un DataFrame. Permet especificar les columnes o un subconjunt d'elles que es consideraran per identificar duplicats mitjançant el paràmetre `subset`.

La opció `keep` controla quina fila es conserva de les duplicades: * `'first'`: guarda la primera aparició (opció per defecte). * `'last'`: guarda la darrera aparició. * `False`: elimina totes les files duplicades.

Amb el paràmetre `inplace = True` es modifica el DataFrame original sense crear una còpia.

```
df.drop_duplicates(subset=['ID Alumne'], keep='first', inplace=True)
```

Aquest codi elimina les files duplicades en el DataFrame `df` basant-se en la columna `'ID Alumne'`. Manté la primera aparició de cada duplicat (`keep='first'`) i modifica el DataFrame original directament (`inplace=True`).

És una manera senzilla i eficient per eliminar duplicats segons una clau específica, assegurant que només queda la primera ocurrència (la primera aparició) de cada valor duplicat.

Ara però, mirem l'exemple anterior amb més detall. Si ordenem les files per algun valor (per exemple, l'ALU-ID) veurem més clarament les dades que contenen els duplicats:

```
[ ]: duplicated_rows = df[df.duplicated(subset=['ALU-ID'], keep=False)]
duplicated_rows.sort_values(by='ALU-ID', ascending=True)
```

```
[ ]:      ALU-ID      Nom_complert  Edat  Alçada  PED  BBDD      SO  DIW  \
18  ALU-1006  Loreto Quesada    18    1.74  NaN   5.8    3.8  6.9
200 ALU-1006  Loreto Quesada    18    1.74  5.7    5.8    3.8  6.9
```

226	ALU-1060	Adán Caparrós	19	1.82	6.1	2.5	7.7	7.5
233	ALU-1060	Adán Caparrós	19	1.82	6.1	2.5	7.7	7.5
26	ALU-1097	Eusebia Boix	20	1.63	7.0	8.2	4.7	4.5
123	ALU-1097	Eusebia Boix	20	1.63	7.0	8.2	4.7	4.5
179	ALU-1112	Felix Bayona	32	1.65	7.1	7.8	4.1	7.4
212	ALU-1112	Felix Bayona	32	1.65	7.1	7.8	4.1	7.4
25	ALU-1142	Macario Galiano	20	1.73	7.3	1.1	10.0	6.1
35	ALU-1142	Macario Galiano	20	1.73	7.3	1.1	10.0	6.1

	Hores_estudi	Faltes_assistència	Grup_classe
18	26.0	1	DAWB
200	26.0	1	NaN
226	9.0	2	DAWB
233	9.0	2	NaN
26	25.0	0	NaN
123	25.0	0	DAMB
179	9.0	25	NaN
212	9.0	25	DAWB
25	11.0	11	DAMA
35	11.0	11	NaN

Simplement podem fer un drop de les files que volem eliminar a partir del seu index.

```
[ ]: df = df.drop([233,26,179,35])
```

```
[ ]: df[df.duplicated(subset=['ALU-ID'], keep=False)]
```

```
[ ]:
      ALU-ID  Nom_complert  Edat  Alçada  PED  BBDD  SO  DIW  \
18  ALU-1006  Loreto Quesada   18   1.74  NaN   5.8  3.8  6.9
200 ALU-1006  Loreto Quesada   18   1.74  5.7   5.8  3.8  6.9
```

	Hores_estudi	Faltes_assistència	Grup_classe
18	26.0	1	DAWB
200	26.0	1	NaN

En aquest cas tenim informació important en les dues còpies, així que podem fer un “merge” dels valors i eliminar una de les còpies.

```
[ ]: df.loc[18]=df.loc[18].combine_first(df.loc[200])
```

```
[ ]: df[df.duplicated(subset=['ALU-ID'], keep=False)]
```

```
[ ]:
      ALU-ID  Nom_complert  Edat  Alçada  PED  BBDD  SO  DIW  \
18  ALU-1006  Loreto Quesada   18   1.74  5.7   5.8  3.8  6.9
200 ALU-1006  Loreto Quesada   18   1.74  5.7   5.8  3.8  6.9
```

	Hores_estudi	Faltes_assistència	Grup_classe
18	26.0	1	DAWB

200 26.0 1 NaN

```
[ ]: df.drop_duplicates(subset=['ALU-ID'], keep='first', inplace=True)
```

Comprovem que no queden duplicats:

```
[ ]: df[df.duplicated(subset=['ALU-ID'], keep=False)]
```

```
[ ]: Empty DataFrame
      Columns: [ALU-ID, Nom_complert, Edat, Alçada, PED, BBDD, SO, DIW, Hores_estudi,
      Faltes_assistència, Grup_classe]
      Index: []
```

Comprovem que la Loreto Quesada tingui tots els valors, tal i com hem forçat:

```
[ ]: df.loc[18]
```

```
[ ]: ALU-ID                      ALU-1006
      Nom_complert               Loreto Quesada
      Edat                        18
      Alçada                      1.74
      PED                         5.7
      BBDD                        5.8
      SO                          3.8
      DIW                         6.9
      Hores_estudi                26.0
      Faltes_assistència           1
      Grup_classe                 DAWB
      Name: 18, dtype: object
```

Ja que hem eliminat algunes files resetegem els índexs pq tornin a ser consecutius i ens ajudin a seleccionar elements:

```
[ ]: df = df.reset_index(drop=True)
      df
```

```
[ ]:        ALU-ID        Nom_complert   Edat   Alçada   PED   BBDD     SO    DIW   \
0    ALU-1115    Cirino Acevedo    25    1.74   7.3    3.7    5.0   4.8
1    ALU-1032    Amada Bermudez   18    1.85   6.7    5.6    3.0   9.1
2    ALU-1214    Pánfilo Diez    23    1.72   6.7    4.8   10.0   6.9
3    ALU-1003    Pastor Reyes    22    1.76   7.0    8.4    3.7   10.0
4    ALU-1068    Estela Escobar   18    1.80   6.4    6.0   10.0   8.0
..        ...                ...        ...        ...        ...        ...
245   ALU-1135    Chuy Herrero    19    1.83   6.8    3.7    7.7    3.5
246   ALU-1002    Martin Palacios   23    1.70   7.4    4.1    6.4    8.0
247   ALU-1225    María Maza    19    1.79   6.5    9.3   10.0   6.4
248   ALU-1247    Josué Conesa    36    1.68   7.7    3.9   10.0   9.2
249   ALU-1221    Dionisio Lobato   21    1.75   6.9   10.0    6.2   10.0
```

	Hores_estudi	Faltes_assistència	Grup_classe
0	15.0	0	DAWA
1	14.0	7	DAWB
2	9.0	19	DAWB
3	20.0	0	DAMA
4	19.0	0	DAMB
..	...	...	...
245	4.0	0	DAMB
246	5.0	10	DAWB
247	2.0	7	DAWA
248	20.0	0	DAWB
249	11.0	3	DAMB

[250 rows x 11 columns]

1.0.3 Seleccionar la informació necessària

En molts casos, no necessitem tota la informació que tenim en els datasets, o la volem tractar de forma diferent. Per tant, l'objectiu es “quedar-nos” amb la informació necessària segons els nostre propòsit.

En el nostre cas, volem saber les mitjes de les notes i també ens interessa poder relacionar les hores d'estudi amb els resultats obtinguts.

Per tant l'edat, l'alçada, ni les faltes d'assistència no seran necessàries (suposem que les notes finals ja no depenen de les faltes d'assistència).

Per eliminar columnes d'un DataFrame amb pandas, s'utilitza el mètode `.drop()`. Per eliminar una sola columna, només cal indicar el nom de la columna, especificant que l'eix a eliminar és `axis=1` o `columns`.

```
[ ]: df.drop('Edat', axis=1, inplace=True)
```

```
[ ]: df
```

```
[ ]:
      ALU-ID  Nom_complert  Alçada  PED  BBDD  S0  DIW  Hores_estudi  \
0    ALU-1115  Cirino Acevedo    1.74  7.3    3.7  5.0  4.8         15.0
1    ALU-1032  Amada Bermudez    1.85  6.7    5.6  3.0  9.1         14.0
2    ALU-1214   Pánfilo Diez    1.72  6.7    4.8 10.0  6.9          9.0
3    ALU-1003   Pastor Reyes    1.76  7.0    8.4  3.7 10.0        20.0
4    ALU-1068  Estela Escobar    1.80  6.4    6.0 10.0  8.0        19.0
..      ...      ...      ...  ...  ...  ...  ...      ...
245  ALU-1135   Chuy Herrero    1.83  6.8    3.7  7.7  3.5          4.0
246  ALU-1002  Martin Palacios    1.70  7.4    4.1  6.4  8.0          5.0
247  ALU-1225   María Maza    1.79  6.5    9.3 10.0  6.4          2.0
248  ALU-1247   Josué Conesa    1.68  7.7    3.9 10.0  9.2        20.0
249  ALU-1221  Dionisio Lobato    1.75  6.9   10.0  6.2 10.0        11.0
```

```
Faltes_assistència  Grup_classe
```


0	0	DAWA
1	7	DAWB
2	19	DAWB
3	0	DAMA
4	0	DAMB
..	...	...
245	0	DAMB
246	10	DAWB
247	7	DAWA
248	0	DAWB
249	3	DAMB

[250 rows x 10 columns]

En el següent exemple eliminem la columna 'Alçada' i 'Faltas_assistència' i en aquest cas, inplace=False indica que el canvi no es fa sobre df directament, sinó que retorna un nou DataFrame amb les columnes eliminades (que assignem a df2).

```
[ ]: df2 = df.drop(['Alçada', 'Faltas_assistència'], axis=1, inplace=False)
```

```
[ ]: df2
```

```
[ ]:
      ALU-ID  Nom_complert  PED  BBDD  SO  DIW  Hores_estudi  \
0  ALU-1115  Cirino Acevedo  7.3   3.7   5.0   4.8           15.0
1  ALU-1032  Amada Bermudez  6.7   5.6   3.0   9.1           14.0
2  ALU-1214  Pánfilo Diez   6.7   4.8  10.0   6.9            9.0
3  ALU-1003  Pastor Reyes   7.0   8.4   3.7  10.0           20.0
4  ALU-1068  Estela Escobar  6.4   6.0  10.0   8.0           19.0
..      ...      ...      ...  ...  ...  ...      ...
245 ALU-1135  Chuy Herrero   6.8   3.7   7.7   3.5            4.0
246 ALU-1002  Martin Palacios  7.4   4.1   6.4   8.0            5.0
247 ALU-1225  María Maza    6.5   9.3  10.0   6.4            2.0
248 ALU-1247  Josué Conesa   7.7   3.9  10.0   9.2           20.0
249 ALU-1221  Dionisio Lobato  6.9  10.0   6.2  10.0           11.0
```

	Grup_classe
0	DAWA
1	DAWB
2	DAWB
3	DAMA
4	DAMB
..	...
245	DAMB
246	DAWB
247	DAWA
248	DAWB
249	DAMB

[250 rows x 8 columns]

1.0.4 Detectar i eliminar valors nuls

Per detectar on hi ha valors nuls:

```
[ ]: df2.isnull()
```

```
[ ]:      ALU-ID  Nom_complert    PED  BBDD    SO    DIW  Hores_estudi  \
0      False      False  False  False  False  False      False
1      False      False  False  False  False  False      False
2      False      False  False  False  False  False      False
3      False      False  False  False  False  False      False
4      False      False  False  False  False  False      False
..      ...      ...    ...    ...    ...    ...      ...
245     False      False  False  False  False  False      False
246     False      False  False  False  False  False      False
247     False      False  False  False  False  False      False
248     False      False  False  False  False  False      False
249     False      False  False  False  False  False      False
```

```
      Grup_classe
0          False
1          False
2          False
3          False
4          False
..           ...
245         False
246         False
247         False
248         False
249         False
```

[250 rows x 8 columns]

Això dona un DataFrame booleà, True on el valor és nul.

Però per saber quants valors nuls hi ha a cada columna, podem fer:

```
[ ]: df2.isnull().sum()
```

```
[ ]: ALU-ID          0
     Nom_complert    0
     PED            4
     BBDD           5
     SO             0
     DIW            0
     Hores_estudi    5
```

```
Grup_classe      0
dtype: int64
```

En el nostre cas hi ha pocs valors nuls però en algun cas ens podria interessar eliminar-los:

Per eliminar totes les files que tinguin almenys un valor nul:

```
df = df.dropna()
```

Per eliminar només les columnes amb almenys un valor nul:

```
df = df.dropna(axis=1)
```

Els valors nuls (NaN) en un DataFrame de pandas són ignorats per funcions agregades com mitjana, suma o mediana, amb la qual cosa aquestes operacions es fan només sobre les dades disponibles. Això pot provocar que la mitjana sigui calculada amb menys valors del total esperat, afectant la representativitat del resultat.

Si moltes files tenen valors nuls, pot aparèixer NaN a la sortida quan no hi ha valors vàlids per al càlcul.

Per aquest motiu, és habitual detectar i gestionar els valors nuls abans d'analitzar les dades, ja sigui eliminant files o columnes amb valors nuls, o bé substituint-los per valors com la mitjana o mediana per preservar la integritat dels agregats (ho veurem més endavant).

1.0.5 Fusió i Concatenació de DataFrames

Primer de tot, per fer els exemples separarem les dades en 4 dataframes diferents segons el grup de classe.

```
[ ]: dawA = df2[df2['Grup_classe']=='DAWA']
     dawB = df2[df2['Grup_classe']=='DAWB']
     damA = df2[df2['Grup_classe']=='DAMA']
     damB = df2[df2['Grup_classe']=='DAMB']
```

```
[ ]: dawA.head()
```

```
[ ]:      ALU-ID      Nom_complert  PED  BBDD   SO  DIW  Hores_estudi  Grup_classe
0   ALU-1115   Cirino Acevedo  7.3   3.7   5.0  4.8         15.0         DAWA
5   ALU-1030   Antonio Ripoll  6.5   3.4   4.3  3.5         13.0         DAWA
7   ALU-1132   Ariel Aguirre  7.0   3.4   7.8  7.8         13.0         DAWA
10  ALU-1075  Ambrosio Salgado  6.1   4.0   2.9  2.1          8.0         DAWA
12  ALU-1050   Nuria Abella   8.7  10.0   9.9  6.2         13.0         DAWA
```

```
[ ]: dawA = df2[df2['Grup_classe']=='DAWA'].reset_index(drop=True)
     dawB = df2[df2['Grup_classe']=='DAWB'].reset_index(drop=True)
     damA = df2[df2['Grup_classe']=='DAMA'].reset_index(drop=True)
     damB = df2[df2['Grup_classe']=='DAMB'].reset_index(drop=True)

     dawA.head(5)
```

```
[ ]:      ALU-ID      Nom_complert  PED  BBDD   S0  DIW  Hores_estudi  Grup_classe
0  ALU-1115  Cirino Acevedo  7.3   3.7  5.0  4.8          15.0          DAWA
5  ALU-1030  Antonio Ripoll  6.5   3.4  4.3  3.5          13.0          DAWA
7  ALU-1132  Ariel Aguirre  7.0   3.4  7.8  7.8          13.0          DAWA
10 ALU-1075  Ambrosio Salgado  6.1   4.0  2.9  2.1           8.0          DAWA
12 ALU-1050  Nuria Abella  8.7  10.0  9.9  6.2          13.0          DAWA
```

```
[ ]: daw = pd.concat([dawA,dawB],ignore_index=True)
dam = pd.concat([damA,damB],ignore_index=True)

dam
```

```
[ ]:      ALU-ID      Nom_complert  PED  BBDD   S0  DIW  Hores_estudi  \
0  ALU-1003  Pastor Reyes  7.0   8.4   3.7  10.0          20.0
1  ALU-1047  Aurelio Arévalo  5.6   5.3  10.0   7.8           NaN
2  ALU-1086  Horacio Solís  6.2   6.2  10.0   5.9          25.0
3  ALU-1246  Lorenza Arellano  5.8   NaN   5.5   3.7           6.0
4  ALU-1142  Macario Galiano  7.3   1.1  10.0   6.1          11.0
..      ...      ...      ...      ...      ...      ...
111 ALU-1082  Eleuterio Valenzuela  5.3   5.9  10.0   8.4          16.0
112 ALU-1236  Rafa Berenguer  5.6   7.5   2.4   9.8           8.0
113 ALU-1013  Belen Barba  8.0   8.7   6.5  10.0          16.0
114 ALU-1135  Chuy Herrero  6.8   3.7   7.7   3.5           4.0
115 ALU-1221  Dionisio Lobato  6.9  10.0   6.2  10.0          11.0

      Grup_classe
0      DAMA
1      DAMA
2      DAMA
3      DAMA
4      DAMA
..      ...
111     DAMB
112     DAMB
113     DAMB
114     DAMB
115     DAMB
```

[116 rows x 8 columns]

Fem un petit exemple per explicar el 'join' (concat però per columnes i no per files). Agafarem una secció de damA i el separarem amb dos dataframes amb diferents files.

```
[ ]: damA_porcio = damA[:10]
damA_porcio
```

```
[ ]:      ALU-ID      Nom_complert  PED  BBDD   S0  DIW  Hores_estudi  \
3  ALU-1003  Pastor Reyes  7.0   8.4   3.7  10.0          20.0
```

11	ALU-1047	Aurelio Arévalo	5.6	5.3	10.0	7.8	NaN
16	ALU-1086	Horacio Solís	6.2	6.2	10.0	5.9	25.0
21	ALU-1246	Lorenza Arellano	5.8	NaN	5.5	3.7	6.0
25	ALU-1142	Macario Galiano	7.3	1.1	10.0	6.1	11.0
30	ALU-1104	Jose Francisco Cuesta	7.5	6.1	9.2	10.0	21.0
33	ALU-1215	Primitivo Cobo	5.9	8.0	8.7	6.2	4.0
34	ALU-1218	Andrés Felipe Arrieta	7.0	0.8	1.1	5.8	9.0
37	ALU-1098	Ovidio Torre	6.1	6.8	0.0	7.8	4.0
40	ALU-1049	Ciriaco Godoy	5.9	7.1	6.6	3.2	19.0

Grup_classe	
3	DAMA
11	DAMA
16	DAMA
21	DAMA
25	DAMA
30	DAMA
33	DAMA
34	DAMA
37	DAMA
40	DAMA

```
[ ]: damA_porcio_1 = damA_porcio[['ALU-ID', 'Nom_complert', 'Grup_classe']].copy()
damA_porcio_2 = damA_porcio[['ALU-ID', 'PED', 'BBDD', 'SO', 'DIW',
↪ 'Hores_estudi']].copy()
damA_porcio_2
```

```
[ ]:      ALU-ID  PED  BBDD    SO  DIW  Hores_estudi
3   ALU-1003  7.0   8.4   3.7  10.0           20.0
11  ALU-1047  5.6   5.3  10.0   7.8           NaN
16  ALU-1086  6.2   6.2  10.0   5.9           25.0
21  ALU-1246  5.8   NaN   5.5   3.7           6.0
25  ALU-1142  7.3   1.1  10.0   6.1           11.0
30  ALU-1104  7.5   6.1   9.2  10.0           21.0
33  ALU-1215  5.9   8.0   8.7   6.2           4.0
34  ALU-1218  7.0   0.8   1.1   5.8           9.0
37  ALU-1098  6.1   6.8   0.0   7.8           4.0
40  ALU-1049  5.9   7.1   6.6   3.2           19.0
```

```
[ ]: df_merged = pd.merge(damA_porcio_1, damA_porcio_2, on='ALU-ID', how='inner')
df_merged
```

	ALU-ID	Nom_complert	Grup_classe	PED	BBDD	SO	DIW	\
0	ALU-1003	Pastor Reyes	DAMA	7.0	8.4	3.7	10.0	
1	ALU-1047	Aurelio Arévalo	DAMA	5.6	5.3	10.0	7.8	
2	ALU-1086	Horacio Solís	DAMA	6.2	6.2	10.0	5.9	
3	ALU-1246	Lorenza Arellano	DAMA	5.8	NaN	5.5	3.7	

4	ALU-1142	Macario Galiano	DAMA	7.3	1.1	10.0	6.1
5	ALU-1104	Jose Francisco Cuesta	DAMA	7.5	6.1	9.2	10.0
6	ALU-1215	Primitivo Cobo	DAMA	5.9	8.0	8.7	6.2
7	ALU-1218	Andrés Felipe Arrieta	DAMA	7.0	0.8	1.1	5.8
8	ALU-1098	Ovidio Torre	DAMA	6.1	6.8	0.0	7.8
9	ALU-1049	Ciriaco Godoy	DAMA	5.9	7.1	6.6	3.2

	Hores_estudi
0	20.0
1	NaN
2	25.0
3	6.0
4	11.0
5	21.0
6	4.0
7	9.0
8	4.0
9	19.0

Quan fas un merge amb pandas, l'argument `how` defineix el tipus de join o unió que vols fer entre els dos DataFrames, segons com es combinen les files basant-se en les claus comunes.

Les opcions principals per `how` són:

- `inner`: Fa una unió interna; només retorna les files amb claus comunes als dos DataFrames. És a dir, només on la clau apareix a ambos. Les files que no coincideixin s'exclouen. És el valor per defecte. És útil quan vols només les interseccions.
- `left`: Fa una unió esquerra; retorna totes les files del DataFrame esquerre (primer), combinades amb les del dret si coincideixen, i s'omplen amb NaN on no coincideixin.
- `right`: Fa una unió dreta; similar a `left` però conserva totes les files del DataFrame dret.
- `outer`: Fa una unió externa completa; retorna totes les files de tots dos DataFrames, i omple amb NaN on no hi hagi coincidència en l'altre.

1.0.6 Ordenar Columnes

Per ordenar files en un DataFrame de pandas, s'utilitza principalment la funció `sort_values()`

Per ordenar per una sola columna, passes el nom de la columna a l'argument `'by'`, i per defecte ordena en ordre ascendent:

```
[ ]: damA_porcio.sort_values(by='PED')
```

	ALU-ID	Nom_complert	PED	BBDD	SO	DIW	Hores_estudi	\
11	ALU-1047	Aurelio Arévalo	5.6	5.3	10.0	7.8	NaN	
21	ALU-1246	Lorenza Arellano	5.8	NaN	5.5	3.7	6.0	
33	ALU-1215	Primitivo Cobo	5.9	8.0	8.7	6.2	4.0	
40	ALU-1049	Ciriaco Godoy	5.9	7.1	6.6	3.2	19.0	
37	ALU-1098	Ovidio Torre	6.1	6.8	0.0	7.8	4.0	

16	ALU-1086	Horacio Solís	6.2	6.2	10.0	5.9	25.0
3	ALU-1003	Pastor Reyes	7.0	8.4	3.7	10.0	20.0
34	ALU-1218	Andrés Felipe Arrieta	7.0	0.8	1.1	5.8	9.0
25	ALU-1142	Macario Galiano	7.3	1.1	10.0	6.1	11.0
30	ALU-1104	Jose Francisco Cuesta	7.5	6.1	9.2	10.0	21.0

Grup_classe	
11	DAMA
21	DAMA
33	DAMA
40	DAMA
37	DAMA
16	DAMA
3	DAMA
34	DAMA
25	DAMA
30	DAMA

```
[ ]: damA_porcio.sort_values(by='PED',ascending=False)
```

```
[ ]:
      ALU-ID      Nom_complert  PED  BBDD    SO  DIW  Hores_estudi  \
30  ALU-1104  Jose Francisco Cuesta  7.5   6.1   9.2  10.0        21.0
25  ALU-1142      Macario Galiano  7.3   1.1  10.0   6.1        11.0
34  ALU-1218  Andrés Felipe Arrieta  7.0   0.8   1.1   5.8         9.0
3   ALU-1003      Pastor Reyes    7.0   8.4   3.7  10.0        20.0
16  ALU-1086      Horacio Solís    6.2   6.2  10.0   5.9        25.0
37  ALU-1098      Ovidio Torre    6.1   6.8   0.0   7.8         4.0
40  ALU-1049      Ciriaco Godoy    5.9   7.1   6.6   3.2        19.0
33  ALU-1215      Primitivo Cobo    5.9   8.0   8.7   6.2         4.0
21  ALU-1246      Lorena Arellano    5.8  NaN   5.5   3.7         6.0
11  ALU-1047      Aurelio Arévalo    5.6   5.3  10.0   7.8        NaN
```

Grup_classe	
30	DAMA
25	DAMA
34	DAMA
3	DAMA
16	DAMA
37	DAMA
40	DAMA
33	DAMA
21	DAMA
11	DAMA

Inclús podem ordenar per més d'una columna:

```
[ ]: df.sort_values(by=['BBDD', 'PED','SO'], ascending=[False, False,False])
```

```
[ ]:      ALU-ID      Nom_complert  Alçada  PED  BBDD      SO  DIW  Hores_estudi  \
195  ALU-1133  Ricardo Redondo    1.76  8.7  10.0  10.0  6.7      18.0
12   ALU-1050    Nuria Abella    1.65  8.7  10.0  9.9   6.2      13.0
190  ALU-1240  Cristian Morell    1.71  7.9  10.0  5.5  10.0      7.0
180  ALU-1081   Juanita Román    1.73  7.8  10.0  10.0  10.0     17.0
67   ALU-1134   César Meléndez    1.61  7.3  10.0  7.5   9.0     14.0
..   ...
127  ALU-1164   Perlita Milla    1.78  7.9   NaN  10.0  5.5     29.0
150  ALU-1072   Godofredo Oliva    1.69  7.3   NaN  10.0  4.1     23.0
210  ALU-1173   Ciriaco Abellán    1.81  6.9   NaN  10.0  2.1     15.0
21   ALU-1246   Lorenza Arellano    1.80  5.8   NaN  5.5   3.7      6.0
184  ALU-1087    Pepe Ródenas    1.74  5.2   NaN  8.8   6.3     16.0

      Faltes_assistència  Grup_classe
195                1      DAMB
12                 22      DAWA
190                25      DAWA
180                 9      DAWA
67                 9      DAWA
..                 ...
127                 0      DAWA
150                24      DAMA
210                12      DAWA
21                 25      DAMA
184                 0      DAWB

[250 rows x 10 columns]
```

1.0.7 Creació de noves columnes

Crear noves columnes en un DataFrame a partir de columnes existents és una operació molt comuna en pandas per enriquir dades i fer càlculs addicionals.

Es poden crear columnes noves amb una assignació directa dels valors calculats:

```
df['nova_columna'] = df['columna1'] + df['columna2']
```

Aquí, cada element de la nova columna és el resultat d'una operació element a element entre les dues columnes existents.

Per fer operacions més complexes com fer la mitjana de diverses columnes en una nova columna es pot usar `.mean(axis=1)` o altres operacions estadístiques:

```
df['mitjana'] = df[['col1', 'col2', 'col3']].mean(axis=1)
```

Això calcula la mitjana per cada fila entre les columnes indiquades.

També es poden crear a partir de funcions aplicades fila a fila amb `.apply()`:

```
df['nova'] = df.apply(lambda row: func(row['col1'], row['col2']), axis=1)
```

On `func` és qualsevol funció que defineixis.


```
[ ]: df2['Mitjana_notes'] = df2[['PED', 'BBDD', 'SO', 'DIW']].mean(axis=1).round(1)
df2.head()
```

```
[ ]:
      ALU-ID      Nom_complert  PED  BBDD    SO  DIW  Hores_estudi Grup_classe \
0  ALU-1115  Cirino Acevedo  7.3   3.7   5.0  4.8         15.0      DAWA
1  ALU-1032  Amada Bermudez  6.7   5.6   3.0  9.1         14.0      DAWB
2  ALU-1214   Pánfilo Diez  6.7   4.8  10.0  6.9          9.0      DAWB
3  ALU-1003   Pastor Reyes  7.0   8.4   3.7 10.0         20.0      DAMA
4  ALU-1068  Estela Escobar  6.4   6.0  10.0  8.0         19.0      DAMB

      Mitjana_notes
0              5.2
1              6.1
2              7.1
3              7.3
4              7.6
```

```
[ ]: df2['Hores_per_nota'] = df2['Hores_estudi'] / df2[['PED', 'BBDD', 'SO', 'DIW']].
      ↪sum(axis=1)
df2.head()
```

```
[ ]:
      ALU-ID      Nom_complert  PED  BBDD    SO  DIW  Hores_estudi Grup_classe \
0  ALU-1115  Cirino Acevedo  7.3   3.7   5.0  4.8         15.0      DAWA
1  ALU-1032  Amada Bermudez  6.7   5.6   3.0  9.1         14.0      DAWB
2  ALU-1214   Pánfilo Diez  6.7   4.8  10.0  6.9          9.0      DAWB
3  ALU-1003   Pastor Reyes  7.0   8.4   3.7 10.0         20.0      DAMA
4  ALU-1068  Estela Escobar  6.4   6.0  10.0  8.0         19.0      DAMB

      Mitjana_notes  Hores_per_nota
0              5.2         0.721154
1              6.1         0.573770
2              7.1         0.316901
3              7.3         0.687285
4              7.6         0.625000
```

```
[ ]: def nota_seleccio(row):
      # PED compta doble
      return (2 * row['PED'] + 2*row['BBDD'] + row['SO'] + row['DIW']) / 6

df2['Nota_seleccio'] = df2.apply(nota_seleccio, axis=1).round(1)

df2.head()
```

```
[ ]:
      ALU-ID      Nom_complert  PED  BBDD    SO  DIW  Hores_estudi Grup_classe \
0  ALU-1115  Cirino Acevedo  7.3   3.7   5.0  4.8         15.0      DAWA
1  ALU-1032  Amada Bermudez  6.7   5.6   3.0  9.1         14.0      DAWB
2  ALU-1214   Pánfilo Diez  6.7   4.8  10.0  6.9          9.0      DAWB
```

3	ALU-1003	Pastor Reyes	7.0	8.4	3.7	10.0	20.0	DAMA
4	ALU-1068	Estela Escobar	6.4	6.0	10.0	8.0	19.0	DAMB

	Mitjana_notes	Hores_per_nota	Nota_seleccio
0	5.2	0.721154	5.3
1	6.1	0.573770	6.1
2	7.1	0.316901	6.6
3	7.3	0.687285	7.4
4	7.6	0.625000	7.1

1.0.8 Pivot Tables i Grouby

Les Pivot tables i els groupby en pandas són dues funcionalitats fonamentals per a l'anàlisi i agregació de dades que ofereixen diferents maneres de resumir i reestructurar un DataFrame.

groupby Permet agrupar dades segons una o diverses columnes, i aplicar funcions d'agregació (com suma, mitjana, comptatge, etc) a cada grup. Retorna un objecte agrupat que es pot transformar en un DataFrame resumit.

És molt útil per obtenir estadístiques agregades per categories.

```
[ ]: df2.groupby('Grup_classe')['Hores_estudi'].mean()
```

```
[ ]: Grup_classe
DAMA    13.357143
DAMB    15.017241
DAWA    14.189655
DAWB    14.726027
Name: Hores_estudi, dtype: float64
```

```
[ ]: df2.groupby('Grup_classe')['Hores_estudi'].sum()
```

```
[ ]: Grup_classe
DAMA    748.0
DAMB    871.0
DAWA    823.0
DAWB   1075.0
Name: Hores_estudi, dtype: float64
```

Perquè es pugui representar sempre s'han de 'resetejar' els índexs:

```
[ ]: df2.groupby('Grup_classe')['Hores_estudi'].mean().reset_index()
```

```
[ ]:   Grup_classe  Hores_estudi
0      DAMA      13.357143
1      DAMB      15.017241
2      DAWA      14.189655
3      DAWB      14.726027
```

Pivot table

És més flexible que groupby i permet crear una taula dinàmica tipus full de càlcul on es defineix:
* Índex (files) * Columnes * Valors a agregar

Permet aplicar diferents funcions d'agregació directament i controlar valors nuls o múltiples columnes.

Molt similar a les *Pivot Tables* d'Excel.

```
[ ]: pivot = pd.pivot_table(df2,
                             index='Grup_classe',
                             values='Nota_seleccio',
                             aggfunc='mean')
```

pivot

```
[ ]:          Nota_seleccio
Grup_classe
DAMA          6.083636
DAMB          6.349123
DAWA          6.664286
DAWB          6.346575
```

```
[ ]: groupedby = df2.groupby('Grup_classe')['Nota_seleccio'].mean().reset_index()
```

groupedby

```
[ ]:  Grup_classe  Nota_seleccio
0      DAMA      6.083636
1      DAMB      6.349123
2      DAWA      6.664286
3      DAWB      6.346575
```